

Alignment Practical Exercises

Benjamin Redelings

Table of Contents

[1. Exercise \(Setup, MAFFT, HoT, Seaview\)](#)

- [1.1. Make a ~/bin directory \(for programs\) and add it to your PATH](#)
- [1.2. Download the alignment files we will use](#)
- [1.3. Install fasta-flip.pl to ~/bin](#)
- [1.4. MAFFT and HoT and Seaview](#)

[2. Exercise \(Iterative refinement, FSA, PRANK\)](#)

- [2.1. Refinement in MAFFT](#)
- [2.2. Alignment with FSA](#)
- [2.3. Alignment with PRANK](#)

[3. BALi-Phy](#)

- [3.1. Creating a script file](#)
- [3.2. Command line options](#)
- [3.3. Running an analysis](#)
- [3.4. Monitoring an analysis](#)
- [3.5. Interpreting parameter names](#)
- [3.6. Summarizing the results of a run](#)
- [3.7. Comparing to MAFFT, FSA, and PRANK](#)

This tutorial assumes a UNIX command line. On Linux or Mac this is built in; on Windows install Cygwin.

1. Exercise (Setup, MAFFT, HoT, Seaview)

1.1. Make a ~/bin directory (for programs) and add it to your PATH

```
% cd // go to your home directory
% pwd // where are we? = print working directory
% ls // look around
% mkdir ~/bin // make a subdirectory
% ls .bash_profile .profile // do you have .profile or .bash_profile?
% nano .profile // edit the startup script
OR
% nano .bash_profile // edit the startup script
<Add the line "PATH=~/bin:$PATH".>
% cat ~/.bash_profile // Check if your changes are there
% cat ~/.profile // Check if your changes are there
<Log out and log back in.>
% echo $PATH // check that ~/bin is in PATH
% echo $PATH | sed 's/:/\n/g' // easier to read!
```

1.2. Download the alignment files we will use

```
% cd
% mkdir alignment_files
% cd ~/alignment_files
% wget http://www.bali-phy.org/examples.tgz // Download the files - Linux
% curl -O http://www.bali-phy.org/examples.tgz // Download the files - Mac
<Alternatively, download the files using your browser>
% tar -xzf examples.tgz // extract the compressed archive
% ls examples // look inside the new directory that was created
% less examples/ferns/cleaned.fasta // examine the downloaded files – type q to quit
```

```
% alignment-info examples/ferns/cleaned.fasta
```

1.3. Install fasta-flip.pl to ~/bin

```
% cd ~/alignment_files/examples
% chmod +x fasta-flip.pl           // make the file executable
% cp fasta-flip.pl ~/bin           // copy the file to ~/bin
% which fasta-flip.pl              // check to make sure its in your PATH
```

1.4. MAFFT and HoT and Seaview

```
% cp ferns/cleaned.fasta ferns.fasta           // make a copy of a file
% mafft --auto ferns.fasta > ferns-mafft.fasta  // try a simple sequence alignment
% fasta-flip.pl ferns.fasta | mafft --auto - | fasta-flip.pl > ferns-mafft-flipped.fasta
% aliview ferns-mafft.fasta & aliview ferns-mafft-flipped.fasta & // take a look at the two good alignments
% ./compare-alignments ferns-mafft.fasta ferns-mafft-flipped.fasta > ferns-compare.html
% firefox ferns-compare.html                  // load in firefox, then shrink the font
```

2. Exercise (Iterative refinement, FSA, PRANK)

2.1. Refinement in MAFFT

Look at the time to perform different numbers of iterations with MAFFT:

```
% cd ~/alignment_files/examples
% time mafft --localpair --maxiterate 0 ferns.fasta > ferns-mafft-0.fasta // No refinement
% time mafft --localpair --maxiterate 1 ferns.fasta > ferns-mafft-1.fasta // Refine 1 times
% time mafft --localpair --maxiterate 1000 ferns.fasta > ferns-mafft-2.fasta // Refine until done
% aliview ferns-mafft-0.fasta & aliview ferns-mafft-1.fasta & // take a look at the two alignments
% ./compare-alignments ferns-mafft-0.fasta ferns-mafft-1.fasta > ferns-compare-0-1.html
% firefox ferns-compare-0-1.html
```

2.2. Alignment with FSA

Lets run FSA with no penalty for wrong homologies, like ProbCons.

```
% time fsa --fast --gapfactor 0 ferns.fasta > ferns-fsa0.fasta // This is similar to probcons – few gaps.
% alignment-info ferns-fsa0.fasta | grep columns
% aliview ferns-fsa0.fasta &
```

Let's run FSA with the standard penalty for wrong homologies:

```
% time fsa --fast ferns.fasta > ferns-fsa.fasta // This is the default. Some gaps.
% alignment-info ferns-fsa.fasta | grep columns
% aliview ferns-fsa.fasta &
```

How long is the probcons-like alignment, versus the FSA alignment?

2.3. Alignment with PRANK

Now let's run PRANK:

```
% time prank -d=ferns.fasta +F -o=ferns-prank
% cp ferns-prank.best.fas ferns-prank.fasta
% alignment-info ferns-prank.fasta | grep columns
% aliview ferns-prank.fasta &
```

How long is the PRANK alignment? How is the shape different from the other alignments?

3. BALi-Phy

This exercise offers a quick overview of bali-phy with an emphasis on concrete analysis of real datasets. Other documentation includes:

- The [Users Guide](#) is much more complete.
- The [Tutorial](#) is similar to this exercise, but covers more functionality.

- You can run **tool --help** to see command line options for *tool*.
- You can run **man tool** to see the manual page for *tool*.
- Manual pages for bali-phy and tools are also available [online](#).

3.1. Creating a script file

First, go to the `~/alignment_files/examples` directory and create a text file called `ferns-intron-exon.txt` that looks like this:

```
# sequence data for 7 partitions
align = ferns.fasta:5-8      # exon 1
align = ferns.fasta:9-453   # intron 1
align = ferns.fasta:454-596 # exon 2
align = ferns.fasta:597-864 # intron 2
align = ferns.fasta:865-948 # exon 3
align = ferns.fasta:949-1328 # intron 3
align = ferns.fasta:1329-1393 # exon 4

smodel = gtr+Rates.gamma+inv # substitution model

name = ferns-intron-exon     # determines name for output directory
```

It should be possible to copy and paste this text into the nano editor below

```
% cd ~/alignment_files/examples
% nano ferns-intron-exon.txt           // Note the instructions on the bottom of the terminal: ^X means Control-X
% cat ferns-intron-exon.txt
% bali-phy -c ferns-intron-exon.txt --test | less
```

3.2. Command line options

A script file can be turned into a command-line by replacing commands like **command = value** with **--command value**. The exception is that for **align=file** commands, you can just write the filename by itself:

```
% bali-phy ferns.fasta:{5-8,9-453,454-596,597-864,865-948,949-1328,1329-1393} --smodel gtr+Rates.gamma+inv --name=ferns-intron-exon --test
```

However, with partitioned analyses the command-line can become kind of long. Raw command lines are more useful with a simple unpartitioned analysis:

```
% bali-phy ferns.fasta --test
```

You can see the options for bali-phy by running:

```
% bali-phy help
```

You can also get help on particular options or models with the help command:

```
% bali-phy help smodel           // help on option
% bali-phy help gtr              // help on model
```

3.3. Running an analysis

After you get the **--test** command to work, delete **--test** and start two copies of this analysis:

```
% bali-phy -c ferns-intron-exon.txt &
% bali-phy -c ferns-intron-exon.txt &
```

These should create directories called `ferns-intron-exon-1` and `ferns-intron-exon-2`. Lets look inside one of these directories:

```
% ls ferns-intron-exon-1
% less ferns-intron-exon-1/C1.log           // numerical parameter samples
% less ferns-intron-exon-1/C1.trees        // tree samples
% less ferns-intron-exon-1/C1.P1.fastas    // alignment samples for partition 1
% less ferns-intron-exon-1/C1.P2.fastas    // alignment samples for partition 2
```

3.4. Monitoring an analysis

You can monitor an analysis in two ways. One way is to use the program **tracer**.

```
% tracer ferns-intron-exon-1/C1.log ferns/intron-exon-2/C1.log // you may need to double-click on Tracer and load log files via the menu.
```

We would like to see that the likelihood and posterior probability have stabilized. Another method of monitoring is just to generate the summary report.

3.5. Interpreting parameter names

The parameter estimates have names that look like context1/value or context1/model:parameter. For example S1/gtr:pi[A] indicates

- S1** The 1st substitution model
- gtr** The GTR (general, time-reversible) nucleotide substitution model.
- pi** The nucleotide frequencies
- [A]** The frequency for A (adenine).

We can get information on what a parameter means by running **bali-phy help model**:

```
% bali-phy help gtr
```

For more information on parameter names, see [the section on Output / Field names](#) in the Users Guide.

3.6. Summarizing the results of a run

```
% bp-analyze ferns-intron-exon-1 ferns-intron-exon-2
% firefox Results/index.html
```

Let's concatenate the summary estimates for each partition to recover an alignment for the whole gene region.

```
% alignment-cat Results/P{1..7}-max.fasta > ferns-bali-phy.fasta
```

Since bali-phy is still running, we can get information on its progress by rerunning **bp-analyze**.

3.7. Comparing to MAFFT, FSA, and PRANK

We can see how the different aligners affect the total alignment length:

```
% alignment-info ferns-bali-phy.fasta | grep columns
% alignment-info ferns-prank.fasta | grep columns
% alignment-info ferns-fsa.fasta | grep columns
% alignment-info ferns-fsa0.fasta | grep columns
% alignment-info ferns-mafft.fasta | grep columns
```

Let's visually compare the alignment by loading all the alignments in aliview. Shrink each alignment so that it is very small, and look at the intron between exons 2 and 3.

```
% aliview ferns-bali-phy.fasta &
% aliview ferns-prank.fasta &
% aliview ferns-fsa.fasta &
% aliview ferns-fsa0.fasta &
% aliview ferns-mafft.fasta &
```